가장 중요한 설계 원칙

Spring이 통제하고, Python + memvid가 검색한다

우리 제품의 통제 플레인 = Spring Boot

우리의 검색 계층은 memvid 기반 사건 메모리입니다. Spring이 사용자·로펌·사건 권한을 확정하고, Python에는 승인된 작업 범위만 전달합니다. 저장·검색 기능에 권한·예산·감사 체계를 결합합니다.

핵심 책임	Spring Boot 통제 → Python 실행 → 결과 확인
01 사건 데이터 접근	로그인·담당 사건 권한을 확인하고 서버 매핑으로 사건별 memvid 파일을 선택합니다. 파일명·사건 ID를 클라이언트가 보냈다는 이유만으로 접근을 허용하지 않습니다.
02 안전한 실행·검토	PII 처리와 입출력 정책을 적용하고, Python에서도 시간·반복·호출 상한을 집행합니다. 결과는 JSON·출처를 검증해 변호사 검토 상태로 저장합니다. 환각 제거를 보장하지는 않습니다.
03 비용·운영 관리	최종 프롬프트 기준으로 예산을 승인·예약하고, 임베딩·중간 호출·재시도까지 사용량을 정산합니다. 관리자 화면에는 확정·추정·진행 중 비용을 구분합니다.

수정안 2: **외부 벡터 DB 중심 검색을 memvid 기반 사건 메모리·검색으로 교체**합니다. 임베딩은 로컬·외부 API 중 품질과 예산에 따라 선택하고, LLM은 선택된 근거로 초안을 생성합니다.

설계 목표이며 구현 완료 선언이 아닙니다. Spring은 우리 프로젝트의 선택이며 Python으로도 통제 계층을 구현할 수 있습니다. 파일 분리는 완전한 물리적 격리, 경량 검색은 운영비 0원을 뜻하지 않습니다.

왜 두 언어로 나누나: Spring + FastAPI·memvid

한 언어로도 구현할 수 있지만, 우리 설계는 업무 통제와 AI 실험의 변경 주기를 나눈다. **Spring Boot·Java 21은 사람·권한·업무 기록을, FastAPI·Python은 memvid 검색·임베딩·모델 작업을 담당한다.**

맡는 일	Spring Boot (Java 21)	FastAPI (Python) + memvid
잘하는 일	로그인·사건 권한·트랜잭션·감사 기록·업무 처리. Virtual Threads는 대기 작업 처리에 활용	memvid 사건 메모리·검색, 임베딩 연동, 모델 교체·데이터 실험. LangChain은 필요할 때 선택
바뀌는 속도	권한·업무 규칙은 일관성과 변경 검토를 중시	모델·프롬프트·검색 설정은 별도 평가 후 빠르게 교체
실패했을 때	중복 처리·잘못된 권한·원장 불일치를 막도록 검증과 트랜잭션 적용	시간·반복·출력 상한으로 중단하고 실패 상태와 사용량 반환. 가드·마스킹·사후 검사 적용
수정안 2의 역할	상담 접수·사건 배정·세션 정책·예산 승인·보고서·관리자 화면	1차 스크리닝·memvid 근거 검색·임베딩·LLM 호출·JSON 검증·사용량 보고

이미지의 기존 Python 역할 중 'RAG' 표기는 수정안 2에서 **'memvid 기반 사건 메모리·검색'**으로 바꾼다. 기존 저장소명이 legal-rag-service여도 문서 표기 변경과 코드·저장소 이름 변경은 별개다. 이번 작업은 문서 개정이다.

Python의 오류가 모두 가볍다는 뜻은 아니다. 개인정보 전송과 유료 API 호출은 되돌리기 어려울 수 있으므로 실행 전 승인과 Python 내부 제한을 함께 적용한다. 역할 분리는 언어의 우열이 아닌 우리 팀의 운영 선택이다.

격리의 의미: 사건별 파일 선택과 접근 통제

단일 파일은 저장 형식이다. 사용자·로펌·사건의 권한 정책은 별도로 적용한다. 여러 사건을 같은 knowledge.mv2에 적재하고 권한 제한 없이 검색하면 다른 사건 자료가 결과에 섞일 수 있다.

순서	우리 설계의 처리	막으려는 오류
1. Spring 권한 확정	인증 정보와 DB로 로펌·담당자·사건 접근 권한 확인	사건 ID를 안다는 이유로 접근 허용
2. 서버 매핑 선택	승인된 사건을 전용 case_1234.mv2에 연결	사용자가 보낸 경로를 그대로 열기
3. Python 범위 집행	인증된 내부 요청 확인, 허용된 저장 루트·사건 매핑만 접근	내부 API 직접 호출·경로 조작으로 우회
4. 결과·주변 데이터 통제	출처의 사건 일치 검사, 캐시·로그·임시 파일·백업도 범위 관리	파일 밖 복사본 또는 캐시를 통한 누출

.mv2는 Python 실행 코드가 아니라 메모리 데이터 파일이다. 내부 요청에 임의 파일 경로를 실어 보내기보다 사건 ID와 검증 가능한 작업 범위를 전달하고 서버가 매핑을 해석하도록 설계한다.

공식 memvid에는 ACL 기능도 있다. 애플리케이션이 권한 문맥을 만들고 실제 차단 설정을 적용해야 하므로 로그인·업무 권한을 자동으로 알아내는 기능과 구분한다. 공식 ACL 설명

사건별 파일은 접근 경계를 명확히 하는 수단이다. 동일 서버의 파일 분리를 별도 하드웨어 수준의 물리적 격리나 완전한 보안 보장으로 표현하지 않는다. 암호화와 키 관리는 별도 적용한다. 로펌당 파일 하나를 선택해도 로펌 내부의 사건별 권한은 여전히 필요하다.

임베딩의 의미: 저장소는 memvid, 모델은 선택 가능

저장 엔진과 임베딩 모델은 서로 다른 선택이다. memvid 파일을 사용하면서 로컬 또는 외부 API에서 만든 임베딩을 사용할 수 있다. 외부 벡터 DB를 줄인다고 임베딩까지 반드시 소형 로컬 모델로 고정할 필요는 없다.

구성 요소	무엇을 결정하나	우리 프로젝트에서 확인할 점
memvid	문서·벡터·색인의 보관과 조회	재개방·동시 접근·복구·사건 범위
임베딩 모델	문장과 질문을 숫자 벡터로 표현	한국어·법률 표현·부정 문장·비용
검색·생성 정책	근거 선택·정렬·프롬프트 구성	누락·오검색·출처 연결·검토 시간

기본 BGE-small-en-v1.5는 영어 모델로 안내된다. 크기가 작다는 이유만으로 모든 질문에서 더 큰 유료 모델보다 반드시 나쁘다고 단정할 수는 없다. 한국어 인테이크 질문과 정답 문서로 비교한다. 프로젝트의 벤치마크도 우리 법률 자료의 성능 보증으로 옮기지 않는다.

Rust 코어의 api_embed는 기능 플래그다. Python SDK에서는 임베딩 제공자·모델 설정을 사용한다. 공식 문서에는 OpenAI와 Gemini 등의 연동 예제가 있지만, 제공자마다 지원 모델과 설정이 다르므로 도입 버전에서 확인한다. Rust 기능 안내, 임베딩 제공자 안내

외부 API를 선택하면 **마스킹·외부 전송 승인 → 임베딩 예산 예약 → 문서·질문에 같은 모델 적용 → 결과 저장·사용량 정산**의 순서로 통제한다. 외부 임베딩도 문서와 질문을 공급자에게 전송하는 경로다. 모델이 바뀌면 동일 차원이라도 기존 벡터와 섞지 않고 재색인을 검토한다.

핵심: **저장 엔진은 경량화하고, 접근 통제는 강화하며, 임베딩은 품질 평가와 예산에 따라 선택한다.** 고성능 모델을 사용해도 검색 정확도 저하 가능성이 완전히 사라지는 것은 아니다.

memvid 혼자 공부하기

가로형 다크 태블릿 교과서

TOP 전체 지도 → MIDDLE 내부 구조 → BOTTOM 실제 코드

쉽게 말하면

사건 서류를 파일에 보관하고, 질문에 필요한 문단을 찾아 근거와 함께 보여주는 방법을 배웁니다. Spring은 권한과 정책을 통제하고, Python은 검색과 모델 작업을 수행합니다.

이번 교과서에 담은 것	읽으면서 확인할 것
전체 학습서 + 흐름도 5개 + 실습 코드 3개	개념 → 처리 단계 → 변수와 함수 순서로 읽기
변호사를 위한 3가지 가치	사건 접근 관리 / 출처 확인 / 지출 관리
참고 이미지 스타일 반영	짙은 배경 / 노란 설명 / 색상별 처리 흐름

가상 인테이크 자료로 학습합니다. 흐름도는 제안 설계이며 구현 완료 증거가 아닙니다. 실습 SDK의 설치·동작은 미검증입니다.

학습 목차

버튼을 누르면 해당 장으로 이동합니다. PDF 책갈피에서도 장을 선택할 수 있습니다.

TOP / 큰 지도

전체 구조 | 사건 격리 | 근거 검증 | 비용 제어 | 두 언어 역할 | 격리의 의미 | 임베딩 선택

기초 / 개념과 실습

먼저 읽을 핵심 결론 | 올려주신 비교표를 이렇게 고쳐 이해하자 | 비유로 이해하는 전체 구조 | v1 예제와 v2 예제를 섞지 말자 | Windows에서 시작하기 | 첫 실습: 가상 문서 3개 저장 | 두 번째 실습: 프로그램을 다시 열어 검색 | 토큰 다이어트: 자동 마법이 아니라 선택... | JSON은 내용의 진실성을 보증하지 않는다 | 검색 종류를 분리해서 이해하자 | 선택 실습: 의미 검색 | 문서 분할은 왜 필요한가 | 공식 저장소의 실제 코드를 읽는 방법 | 사건별 파일 격리를 제대로 이해하기

MIDDLE / 우리 시스템과 도입 가치

우리 Spring Boot–FastAPI... | 비용 관리 여섯 가지를 연결하는 연습 | 혼자 하는 검색 품질 평가 | 오류가 나면 확인할 순서 | 변호사에게 설명할 3가지 가치

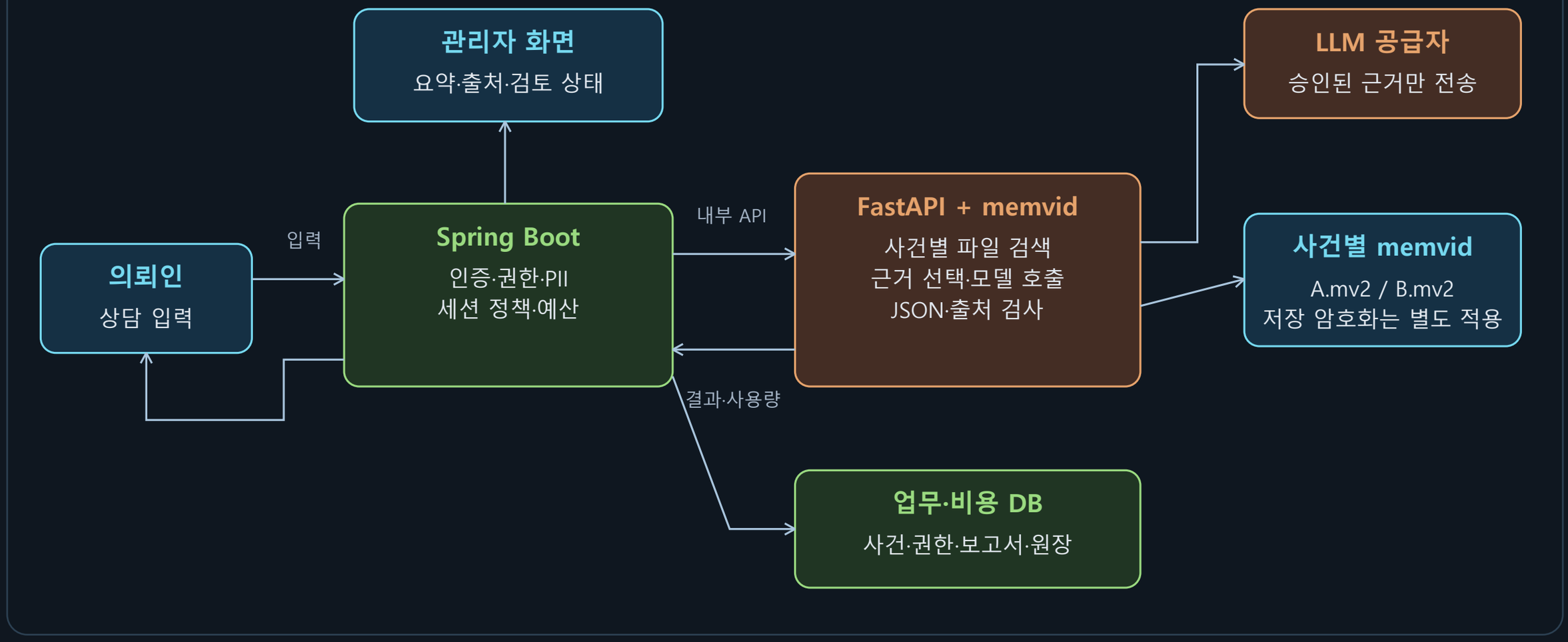
BOTTOM / 복습과 코드

7일 자습 일정 | 이해 확인 문제와 답 | 다음에 설계 수정안 2로 옮길 문장 | 직접 확인한 참고자료 | 코드 01 저장 | 코드 02 검색 | 코드 03 의미 검색

TOP · 수정안 2 전체 지도

쉽게 말하면

두 서버의 역할과 저장소를 나눠 보세요. memvid는 검색용 기억을, 업무 DB는 사건·권한·보고서·비용 원장을 담당합니다.

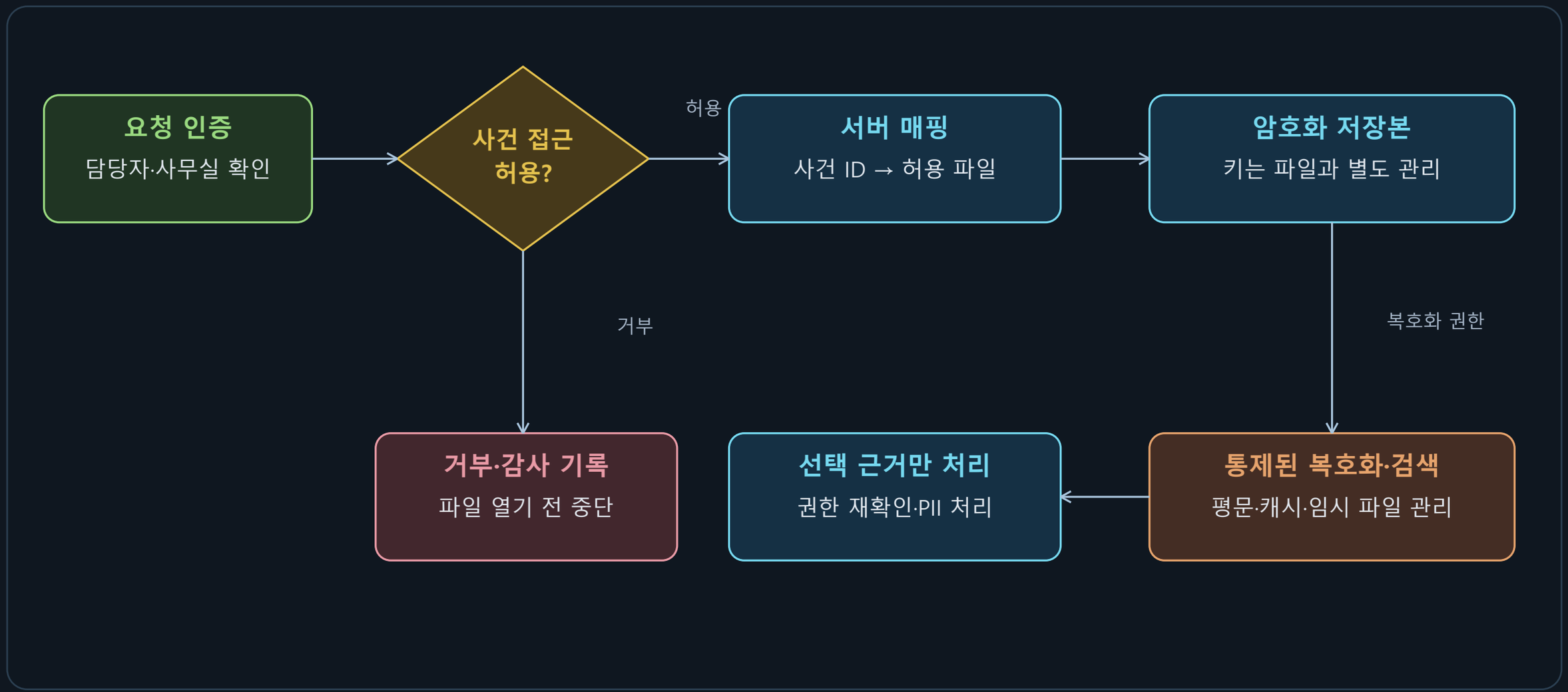


입력은 Spring에서 통제하고 검색은 Python에서 수행합니다. 모델 호출 전 예산 승인과 결과 수신 후 검증을 유지합니다.

MIDDLE · 사건별 파일과 보안 경계

쉽게 말하면

파일을 나누는 것, 암호화하는 것, 읽을 권한을 확인하는 것은 서로 다른 보호 장치입니다.

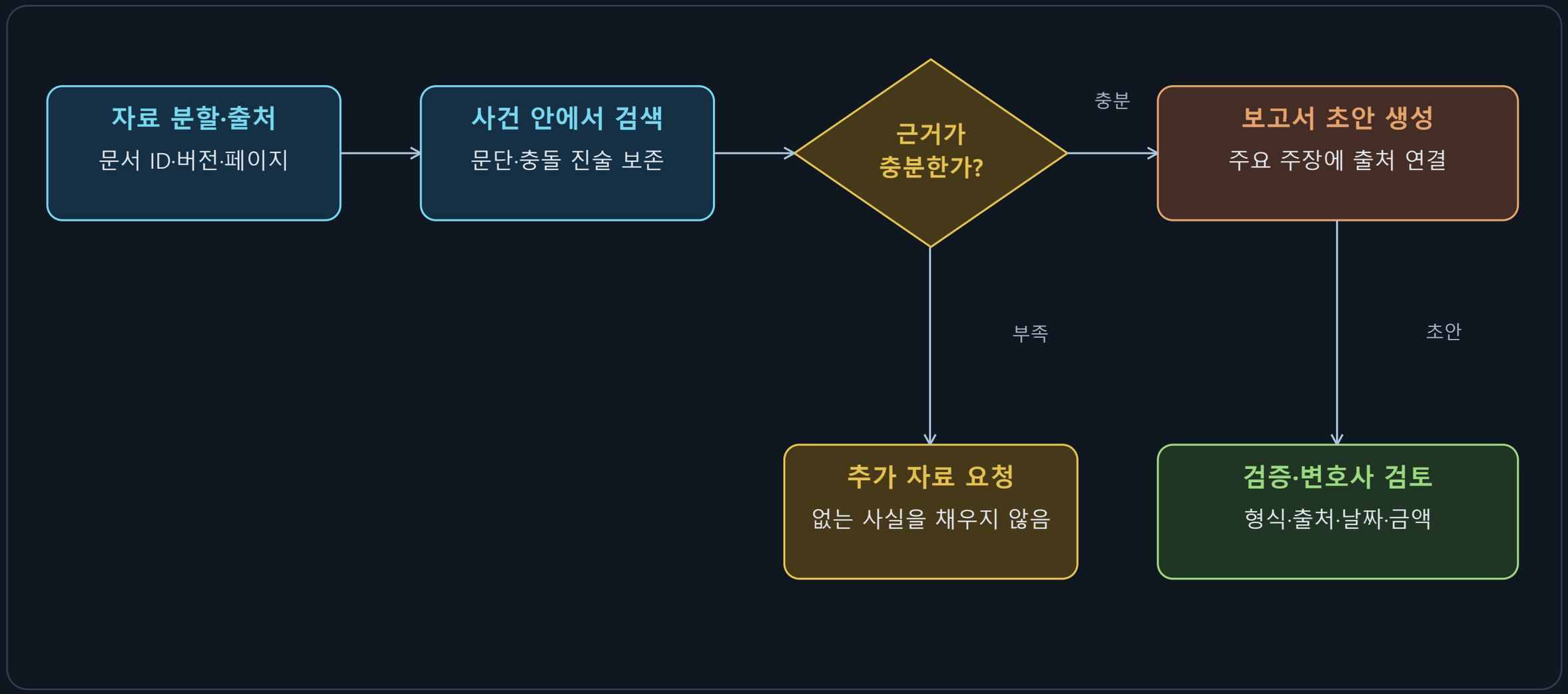


권한이 없으면 파일 열기 전에 중단합니다. 저장 암호화를 적용해도 검색 중 평문과 임시 파일은 별도로 관리해야 합니다.

MIDDLE · 근거에서 검토 가능한 보고서까지

쉽게 말하면

검색 결과가 있다는 것만으로 주장 전체가 증명되는 것은 아닙니다. 원문과 주요 주장을 연결합니다.

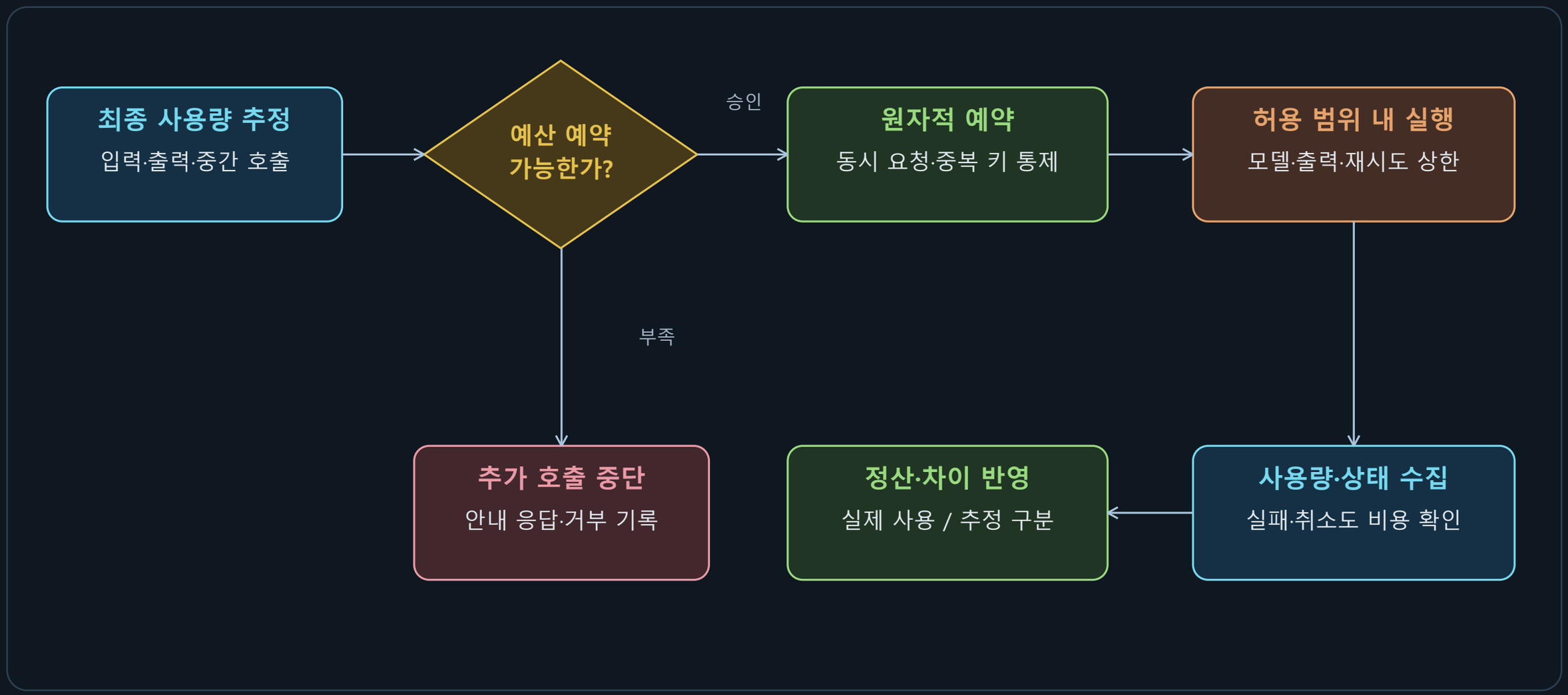


근거 부족은 추가 자료 요청으로 처리합니다. 생성된 JSON의 형식·출처·내용을 각각 검사합니다.

MIDDLE · 예산 승인과 사용량 정산

쉽게 말하면

예산 검사와 예약을 한 번의 원자적 처리로 묶어 동시 요청이 같은 잔액을 중복 승인하지 않게 합니다.



실행 후에는 실패·취소도 사용량을 확인합니다. 비용 미확정 요청의 예약을 즉시 전부 풀지 않습니다.

BOTTOM · 첫 실습의 실행 지도

쉽게 말하면

저장 프로그램을 끝내고 검색 프로그램에서 다시 파일을 엽니다. 이 지속성을 직접 확인하는 것이 첫 목표입니다.



01 → 02를 먼저 완료합니다. Windows 키워드 검색과 선택 의미 검색의 지원 범위를 구분합니다.

1. 먼저 읽을 핵심 결론

이제부터는 기존 학습서의 상세 내용을 순서대로 읽습니다. 그림은 앞의 TOP·MIDDLE·BOTTOM 페이지를 참고하세요.

memvid를 도입하면 별도 벡터 DB 서비스를 운영하는 부담을 줄일 수 있다. 하지만 문서를 찾고 그 근거로 LLM이 답하는 작업 자체는 여전히 RAG다. 우리 프로젝트의 변경을 정확히 표현하면 **"외부 벡터 DB 중심 검색을 memvid 기반 파일 검색으로 교체한다"**가 된다.

사용자가 준 ai-agent-orchestrator/memvid 주소는 이번 조회에서 내용을 확인하지 못했다. 검색으로 확인한 공개 프로젝트는 memvid/memvid다. 두 주소의 관계나 리다이렉트를 확인했다고 가정하지 않았다. 아래 내용은 확인 가능한 공식 프로젝트를 기준으로 한다.

확인 시 GitHub에는 약 17.3k 스타가 표시되어 있었다. 스타는 관심도의 참고자료다. 우리 시스템의 한국어 검색 품질이나 운영 적합성의 검증 결과로 사용하지 않는다. 공식 저장소

2. 올려주신 비교표를 이렇게 고쳐 이해하자

아래는 제품 간 벤치마크 결과가 아니라 도입 판단을 위한 설명이다.

비교 항목	별도 벡터 DB 서비스	memvid 기반 파일 레이어
검색 정확도	임베딩·문서 분할·검색 설정·자료 품질에 좌우된다	같은 요인의 영향을 받는다. 동일 정확도는 평가 후 판단한다
인프라 비용	관리형 요금 또는 직접 운영하는 서버 비용이 들 수 있다	별도 DB 서비스 비용을 줄일 수 있다. CPU·디스크·백업·임베딩·LLM 비용은 남는다
데이터 격리	권한·테넌트 분리·네트워크 설정으로 설계한다	사건별 파일 분리가 이해하기 쉽다. 파일 경로·권한·백업·로그까지 통제해야 한다
배포	서비스 구성과 연결 설정이 필요하다	파일 이동은 간단하다. SDK·모델·영속 디스크 등 실행 환경은 따로 준비한다
확장	많은 사용자와 분산 운영을 위한 선택지가 있다	작은 범위부터 검증하기 좋다. 여러 프로세스가 같은 파일에 쓰는 경우를 따로 검증한다
장애 복구	서비스의 백업·복구 정책이 필요하다	파일을 안전하게 백업하고 복원 가능한지 확인해야 한다

"서버리스"는 여기서 **메모리 검색용 서버를 별도로 띄우지 않는 사용 방식**으로 이해하면 된다. Spring Boot와 FastAPI 서버가 없어지는 것은 아니다. 클라우드 함수에 배포한다면 임시 디스크가 사라지는 문제도 별도로 해결해야 한다.

공식 문서에는 용량 티어와 API 키 사용 시 쿼리 계량 정책도 나온다. 오픈소스 코어, 배포된 SDK, 클라우드 계정의 정책을 한 가지로 뭉뚱그려 "무조건 무료·무제한"이라 적지 않는다. 실제 도입 버전의 정책을 확인한다. 용량 안내, 쿼리 사용량 안내

3. 비유로 이해하는 전체 구조

사무실에서 사건 서류를 찾는 상황으로 생각해 보자.

개발 용어	사무실 비유	예시
원본 문서	계약서·연락 메모·입금 내역	의뢰인이 제출한 서류
청크(chunk)	검색하기 좋게 나눈 문단	계약 종료일이 있는 문단
메타데이터	서류 표지에 붙인 정보	문서 ID, 사건 ID, 페이지
인덱스(index)	서류 위치를 빨리 찾는 색인	보증금 관련 문단 위치
임베딩(embedding)	의미를 숫자 좌표로 표현한 것	반환과 돌려받기의 의미 비교
검색(retrieval)	질문에 맞는 자료를 꺼내는 일	보증금 반환 관련 근거 3개
컨텍스트(context)	LLM이 참고할 자료 묶음	선택된 문단과 출처
생성(generation)	자료를 읽고 보고서를 쓰는 일	상담 준비용 요약 JSON

memvid를 쓰는 실습의 흐름은 다음과 같다.

```
가상 사건 문장
  → 저장용 Python 코드
  → case-demo.mv2
  → 검색용 Python 코드
  → 관련 문장·출처
  → 필요한 경우에만 LLM
  → 검증된 보고서
```

파일에 저장한 내용은 프로그램 종료 후에도 남는다. 이 지속성이 장기 기억의 가장 기본적인 의미다. 저장만 했다고 에이전트가 자동으로 언제 어떤 기억을 꺼낼지 결정해 주는 것은 아니다. 조회 시점과 저장 정책은 애플리케이션에서 정한다.

4. v1 예제와 v2 예제를 섞지 말자

예전 글에서 영상·QR 코드·MP4를 메모리로 사용하는 설명을 볼 수 있다. 현재 공부 대상은 .mv2 형식의 v2다. 코어는 Rust로 재작성되었고 Python에서는 SDK를 통해 이용한다. 옛날 MemvidEncoder 중심 튜토리얼을 현재 코드에 그대로 붙이지 않는다. 공식 변경 이력

구분하는 습관:

1. 설치 이름을 본다: 이번 실습은 memvid-sdk다.

2. import 이름을 본다: 이번 실습은 memvid_sdk다.

3. 파일 확장자를 본다: 이번 실습은 .mv2다.

4. 문서 작성일과 설치된 버전을 기록한다.

pip install의 패키지 이름과 Python의 import 이름이 다를 수 있다. 하이픈이 있는 설치 이름을 그대로 import하면 안 된다.

5. Windows에서 시작하기

이 폴더에 다음 파일을 준비해 두었다.

```
memvid-beginner/
  README.md                  지금 읽는 학습서
  requirements.txt           공부에 사용할 SDK 버전
  01_store.py                가상 자료 저장
  02_search.py               저장 자료 검색 + JSON
  03_semantic_optional.py    Linux/macOS 선택 의미 검색
  data/                      실행하면 만들어지는 메모리 폴더
```

Windows PowerShell에서 아래 명령을 한 줄씩 실행한다. Python이 설치되어 있고 py 명령으로 실행되는 환경을 가정한다.

```
cd C:₩myLectureWs₩study-notes₩memvid-beginner
py --version
py -m venv .venv
.₩.venv₩Scripts₩python.exe -m pip install -r requirements.txt
.₩.venv₩Scripts₩python.exe -m pip show memvid-sdk
```

가상환경은 이 공부에 쓰는 라이브러리를 다른 프로젝트와 분리하는 폴더다. 가상환경을 활성화하지 않고도 위처럼 Python 실행 파일의 경로를 직접 지정할 수 있다.

py가 없다면 Python 설치 상태부터 해결한다. python --version만 동작하는 환경에서는 첫 두 명령의 py를 python으로 바꾸면 된다. SDK는 네이티브 확장을 포함하므로 Python 버전뿐 아니라 운영체제와 CPU에 맞는 배포 파일이 필요한 점도 기억한다. 설치 안내

이번 파일은 확인한 PyPI 배포 버전 2.0.160을 고정했다. "항상 최신"이라는 뜻은 아니다. 재현 가능한 공부를 위한 기준점이다. 다른 버전으로 바꾼다면 실행 결과와 반환 구조를 다시 확인한다. PyPI는 Windows에서 기본 로컬 임베딩을 지원하지 않는다고 안내한다. **Windows 저장·키워드 검색과 Windows 기본 로컬 의미 검색은 지원 범위가 다르다.** PyPI 배포·플랫폼 안내

6. 첫 실습: 가상 문서 3개 저장

실행:

```
.₩.venv₩Scripts₩python.exe 01_store.py
```

이 스크립트는 계약 메모, 연락 메모, 자료 목록을 저장한다. 생성된 data₩case-demo.mv2를 메모장으로 편집하지 않는다. 일반 텍스트 파일이 아니라 SDK가 읽고 쓰는 파일이다.

이제 01_store.py를 열어 다음 순서로 읽는다.

6.1 import와 환경변수

import os는 실행 환경을 다루는 Python 표준 기능을 가져온다. from pathlib import Path는 파일 경로를 다루는 도구를 가져온다. from memvid_sdk import create는 설치한 라이브러리에서 메모리 파일 생성 기능을 가져온다.

코드는 SDK를 가져오기 전에 MEMVID_OFFLINE을 설정하고, 문서를 넣을 때도 enable_embedding=False를 명시한다. 첫 실습은 검색과 저장의 흐름을 분리해서 배우기 위한 것이다. 환경변수만으로 제품 전체의 보안을 증명할 수 있는 것은 아니므로 운영 단계에서는 실제 네트워크 통신도 확인한다.

6.2 경로 만들기

Path(__file__).resolve().parent는 지금 실행하는 스크립트가 들어 있는 폴더를 의미한다. 그래서 다른 폴더에서 실행해도 이 실습 폴더 아래에 데이터를 저장한다.

path.parent.mkdir(parents=True, exist_ok=True)는 저장 폴더가 없으면 만든다. 이미 존재하는 폴더 때문에 실패하지 않도록 옵션을 준 것이다.

6.3 기존 파일 검사

if path.exists()는 파일이 이미 있는지 묻는다. 있으면 SystemExit으로 실행을 끝낸다. 첫 파일은 "한 번 생성하고 다시 열기"를 배우도록 만들었다. 저장 코드를 반복 실행해서 동일 자료를 계속 넣는 것도 막는다.

공식 API에서 create()는 기존 파일을 덮어쓸 수 있다. 새 파일 생성과 기존 파일 열기를 구분하는 것은 이 실습에서 가장 먼저 익힐 습관이다. create(path)와 use("basic", path)의 인자 순서도 다르다. 공식 시작 예제

6.4 documents와 for문

documents는 세 건의 자료를 담은 리스트다. 각 자료에는 문서 ID, 제목, 본문이 있다. for source_id, title, text in documents:는 자료를 하나씩 꺼내 세 변수에 나눠 담는다.

Java의 DTO 목록을 반복하면서 저장하는 코드와 비슷하다. Python에서는 들여쓰기가 코드 블록을 정한다. for 아래에서 들여쓰기한 부분만 반복된다.

6.5 put의 각 입력값

입력값	우리 실습에서의 뜻
title	사람이 읽는 문서 제목
label	자료 종류를 나타내는 라벨
metadata	출처를 추적하기 위한 부가 정보
text	실제로 저장하고 검색할 문장
enable_embedding	이번 문장을 벡터로도 만들지 여부

source_id는 원본 자료를 추적하기 위한 우리 애플리케이션의 ID다. frame_id는 저장 엔진이 사용하는 ID다. 둘을 혼동하지 않는다. 검색 결과와 원본 문서의 연결을 운영에서 유지하려면 별도 출처 레지스트리도 설계한다.

메타데이터에 case_id를 적는 것만으로 다른 사건 검색이 차단되지는 않는다. 이것은 데이터 표식이며 권한 검증과 같은 개념이 아니다.

6.6 with와 seal

with는 작업 범위가 끝났을 때 자원을 정리하는 문법이다. 파일을 여는 코드에서는 핸들을 계속 붙잡고 있지 않도록 돕는다. seal() 은 이 실습의 저장 마무리 단계다. 호출 이후 같은 핸들로 계속 쓰는 코드를 추가하지 않는다. 다음 검색은 파일을 새로 연다.

Python SDK

완료 기준: 저장 완료 메시지와 파일 생성을 확인한다. 다시 실행하면 기존 파일이 있다는 안내가 나와야 한다. 파일이 있다는 사실만으로 내용 복원이 검증된 것은 아니므로 다음 실습까지 진행한다.

7. 두 번째 실습: 프로그램을 다시 열어 검색

```
.₩.venv₩Scripts₩python.exe 02_search.py "보증금"
.₩.venv₩Scripts₩python.exe 02_search.py "송금"
.₩.venv₩Scripts₩python.exe 02_search.py "존재하지않는검색어XYZ"
```

첫 검색은 보증금 관련 메모를, 두 번째는 송금 내역 관련 메모를 찾는지 확인한다. 세 번째는 근거가 없는 질문의 처리를 살핀다. 순위·점수·반환 필드는 실행한 SDK 버전에서 직접 확인한다.

7.1 use는 기존 파일을 연다

use("basic", str(path))는 앞서 만든 파일을 연다. basic은 기본 SDK 어댑터 이름이다. 기초 실습에서 LangChain이나 LlamaIndex를 먼저 배울 필요는 없다.

저장 프로그램과 검색 프로그램이 서로 다르다는 점이 중요하다. 저장할 때 만든 Python 변수는 이미 사라졌지만 파일에서 내용을 다시 찾는다. 이것이 첫 번째 장기 기억 실험이다.

7.2 find의 세 가지 설정

mode="lex"는 단어 중심으로 검색한다. k=3은 결과를 최대 세 개 요청한다. snippet_chars=500은 결과 미리보기 길이에 관한 설정이다. 검색 결과 세 개를 요청했다고 반드시 관련 자료 세 개가 존재하는 것은 아니다.

find()는 근거 검색, ask()는 검색 내용을 이용한 질의응답 기능으로 구분해서 공부한다. 처음부터 ask()만 호출하면 저장·검색 문제와 LLM 생성 문제를 구별하기 어렵다. 조회 기능 안내

7.3 결과를 먼저 그대로 출력하는 이유

result는 SDK 응답이다. 예제는 이것을 JSON으로 먼저 보여준다. ensure_ascii=False는 한글을 읽기 좋게 표시하고, indent=2는 들여쓰기를 준다. default=str은 교육용 출력에서 일부 값을 문자열로 표현하도록 돕는다. 이 옵션이 응답 스키마 검증을 대신하지는 않는다.

공식 문서끼리도 ask()의 답변 필드가 text 또는 answer로 나오거나 검색 미리보기 표현이 다른 부분이 있다. 이 자료는 ask()를 기본 실습에서 제외하고, 검색 응답 원본부터 확인하도록 했다. snippet, preview, text 순의 읽기는 관찰을 돕는 실습 편의 코드다. 운영 코드에서는 고정 버전의 응답 구조에 맞춰 변환기를 명시한다.

7.4 dict, list, get을 읽는 법

result["hits"]는 딕셔너리에서 검색 결과 목록을 꺼낸다. hit.get("title")은 결과 하나에서 제목을 읽는다. 값이 없을 때 오류를 내는 대괄호 접근과 달리 get()은 기본적으로 None을 돌려준다.

for hit in hits:는 결과를 하나씩 처리한다. evidence.append(...)는 선택한 근거를 목록 끝에 추가한다. len(selected)는 선택한 문자열 길이를 센다.

코드를 읽을 때 "지금 다루는 것은 결과 전체인가, 결과 하나인가, 문자열인가"를 계속 구분해 보자. 초보자의 많은 오류는 이 자료형 구분에서 나온다.

8. 토큰 다이어트: 자동 마법이 아니라 선택 정책

두 번째 예제는 근거 텍스트 합계에 1,200글자 상한을 둔다. 이 수치는 실습용이며 권장 운영 값이 아니다. 제목·JSON 구조·시스템 지시문까지 포함한 전체 프롬프트의 토큰 상한도 아니다.

예를 들어 원본이 30,000글자이고 선택 문단이 1,200글자라면, 참고 자료의 글자 수는 줄었다. 하지만 이를 그대로 "토큰 96% 절약"이나 "비용 96% 절감"이라고 기록하면 안 된다. 실제 토큰은 모델의 토크나이저로 세고, 비용은 모든 모델 호출의 사용량으로 계산한다.

검색으로 일부 문장만 고르는 일과 요약 모델을 호출해 내용을 다시 쓰는 일은 다르다. 요약 모델을 추가하면 그 호출도 비용이 든다. 또한 짧게 만들수록 예외 조항·날짜·부정 표현이 빠질 수 있다.

운영용 개선 순서:

1. 출처와 함께 관련 문단을 찾는다.

2. 중복을 제거한다.

3. 날짜·금액·부정 문장·상반되는 진술을 보존한다.

4. 모델 토큰 한도 안에서 문단 단위로 선택한다.

5. 근거가 부족하면 추가 자료를 요청한다.

송금 내역은 아직 제출되지 않았다를 잘라서 송금 내역만 남기면 의미가 변한다. 예제의 단순 글자 자르기는 길이 제한을 이해하기 위한 코드다. 실제 법률 인테이크에서는 문장 경계와 출처 범위를 보존하도록 바꾼다.

비용 통제는 검색 전에 한 번, 최종 프롬프트가 완성된 뒤 다시 한 번 수행한다. 검색으로 입력을 줄였다는 이유로 Spring의 예산 검사를 우회시키지 않는다.

9. JSON은 내용의 진실성을 보증하지 않는다

두 번째 예제의 JSON은 보고서가 아니라 "선택된 검색 근거 묶음"이다. LLM 호출 없이도 JSON을 만들 수 있다. 정해진 모양의 데이터와 사실이 맞는 데이터는 서로 다른 검증 대상이다.

우리 프로젝트의 최종 출력은 다음처럼 설계할 수 있다. 아래는 SDK 응답이 아니라 **우리 서비스가 정의할 출력 계약 예시**다.


```
{
  "case_id": "case-demo",
  "status": "NEEDS_REVIEW",
  "claim_summary": "보증금 반환 관련 상담 요청",
  "evidence": [
    {
      "source_id": "D002",
      "statement": "의뢰인은 2026-10-05 반환을 요청했다고 진술함",
      "verification": "CLIENT_STATEMENT"
    }
  ],
  "timeline": [
    {"date": "2026-10-05", "event": "반환 요청 진술", "source_ids": ["D002"]}
  ],
  "missing_information": ["송금 내역 확인 필요"],
  "requires_lawyer_review": true
}
```

모델에게 JSON을 요구한 뒤에는 Python에서 필수 필드·타입·배열 길이·허용 상태값을 검사한다. 다음으로 출처 ID가 이번 사건의 검색 결과에 실제 존재하는지 검사한다. Spring에서도 수신 DTO와 정책을 검증한다.

CLIENT_STATEMENT는 제출자의 진술이라는 의미다. "제출자가 말했다"와 "증거로 확인되었다"를 분리해야 관리자 화면도 올바른 의미를 전달한다. 법적 결론이나 청구취지는 변호사 검토용 초안으로 표시한다.

10. 검색 종류를 분리해서 이해하자

종류	질문하는 방식	장점	공부할 때 확인할 약점
키워드 검색	같은 단어가 있는가	금액·고유 표현·특정 용어 추적	동의어·표현 차이
의미 검색	비슷한 뜻인가	다른 표현의 관련 문장 탐색	비슷하지만 다른 사안을 섞을 가능성
하이브리드 검색	두 신호를 어떻게 조합할까	정확한 표현과 의미의 상호 보완	실제 결합 방식·순위 설정 확인 필요

mode="auto"라는 이름만 보고 모든 버전에서 항상 같은 하이브리드 결합을 한다고 가정하지 않는다. 벡터 인덱스 존재 여부와 설치 버전의 구현을 확인한다.

보증금과 월세는 주거 계약이라는 맥락이 비슷하지만 같은 돈이 아니다. 의미가 가깝다는 이유만으로 올바른 증거라고 판정할 수 없다. 검색 점수는 법적 타당성이나 답변이 맞을 확률도 아니다.

11. 선택 실습: 의미 검색

03_semantic_optional.py는 기본 로컬 임베딩을 시도하는 Linux/macOS용 학습 코드다. Windows에서는 안내 후 종료하도록 만들었다. WSL을 사용한다면 Linux 쪽 Python과 가상환경을 새로 준비해야 한다. Windows의 .venv를 그대로 가져다 쓰지 않는다.

Linux/macOS에서 이 폴더로 이동한 뒤:

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
.venv/bin/python 03_semantic_optional.py
```

첫 사용에는 임베딩 모델 다운로드가 필요할 수 있다. 이미 MEMVID_OFFLINE=1을 설정한 셸에서는 초기 다운로드를 진행할 수 없으므로 별도 터미널에서 시작한다. 외부 API 키가 있는 경우 코드가 중단되도록 했으며, 이 실습에서 키를 입력할 필요는 없다.

코드의 의도는 영어로 적은 보증금 반환 메모와 표현이 다른 영어 질문을 의미로 연결해 보는 것이다. 기본 bge-small은 공식 문서에서 영어 모델로 안내한다. 이것의 영어 실습 성공이 한국어 법률 문서 성능을 입증하지는 않는다. 임베딩 모델 안내

여기서 배우는 변경점은 세 가지다. 파일을 만들 때 벡터 인덱스를 켜고, 자료를 넣을 때 임베딩을 생성하고, 검색 질문도 같은 임베딩 모델로 처리한다. 파일을 다시 열었을 때도 의미 검색이 되는지 확인한다. 예제는 영어 기본 경로만 다룬다. 외부 임베딩 제공자를 사용할 때는 별도 설정과 통합 검증이 필요하다.

문서 임베딩과 질문 임베딩은 같은 의미 공간에 있어야 한다. 차원 수가 같아도 다른 모델을 임의로 섞으면 안 된다. 모델을 바꿀 때는 기존 원본에서 새 파일을 만들어 전체를 재색인하는 방식을 먼저 배운다.

한국어 단계에서는 "보증금 돌려받기", "임대차 종료 후 반환", "전세금 미지급" 같은 표현 차이를 평가한다. 언어 지원과 우리 자료의 검색 품질을 확인한 뒤 모델을 선정한다. Windows에서 로컬 추론을 유지하려면 외부 로컬 임베딩 프로세스와의 연동도 검토할 수 있지만, 기본 ONNX 지원과는 별도 경로이며 이번 예제가 검증한 내용은 아니다.

12. 문서 분할은 왜 필요한가

한 파일 전체를 한 문장처럼 저장하면 관련 없는 부분까지 함께 검색될 수 있다. 반대로 너무 잘게 자르면 "누가, 언제, 무엇을 부정했는지"가 끊어진다.

실습을 확장할 때는 계약서의 조항이나 사건 메모의 문단 단위로 나눠 보자. 인접 문단 일부를 겹치게 두는 overlap은 앞뒤 맥락을 남기기 위한 방법이다. 특정 청크 크기가 모든 자료에 최적인 것은 아니다.

각 청크에 남길 정보의 예:

```
source_id: DOC-003
chunk_id: DOC-003-P02-C01
case_id: CASE-001
page: 2
document_version: 1
statement_type: CLIENT_STATEMENT
```

원본 PDF를 그대로 SDK에 넣기 전에 누가 텍스트를 추출하고 누가 마스킹하는지 먼저 결정해야 한다. Python으로 원본 PDF가 넘어간 뒤 마스킹하면 "원본 PII는 Python에 보내지 않는다"는 설계와 다르게 된다. PDF 스캔본의 OCR도 별도 문제다.

학습 초기에는 가상 텍스트만 사용한다. 그다음 마스킹된 텍스트 파일, 출처가 보존된 문단, 마지막으로 문서 파싱 순으로 확장하면 문제를 구분하기 쉽다.

13. 공식 저장소의 실제 코드를 읽는 방법

Python만 배워도 memvid를 사용할 수 있다. Rust 코어를 전부 해석할 필요는 없다. 다만 README의 설명과 코드가 어떻게 이어지는지 보면 라이브러리를 더 정확히 이해할 수 있다.

13.1 입구: src/lib.rs

확인한 공개 코드에는 다음 선언이 있다.

```
pub mod lex;
pub mod vec;
```

mod는 기능을 나눈 모듈이다. Java의 패키지와 완전히 같지는 않지만 "관련 코드 묶음"으로 읽어도 첫 공부에는 충분하다. pub는 외부에 공개한다는 뜻이다. 이 두 줄에서는 단어 검색과 벡터 관련 기능이 코드에서도 나뉜다는 점을 확인하면 된다.

같은 파일에서 FileLock, EmbeddedWal, Memvid, SearchRequest 등의 공개 타입도 확인할 수 있다. 이름을 따라가며 잠금·복구·메모리 객체·검색 요청이라는 책임을 구별해 보자. 이름이 존재한다는 사실만으로 모든 장애가 해결된다고 결론 내리지는 않는다. 공개 lib.rs

13.2 구체적인 검색 흐름: src/lex.rs

확인한 LexIndexBuilder는 문서의 내용과 식별 정보를 수집하고 직렬화할 데이터를 만든다. LexIndex 쪽에는 저장된 데이터를 읽어 검색하는 코드가 있다. search에서 질문을 토큰화하고, 후보를 계산하고, 제한 수만큼 결과를 남긴 뒤 미리보기를 구성하는 흐름을 볼 수 있다.

compute_matches 안에서는 문서를 돌며 범위를 확인하고, 일치한 부분을 찾고, 점수순으로 정렬하고, 같은 프레임의 중복을 줄이는 처리가 보인다. 이 파일만 보고 SDK의 모든 검색 경로가 이 구현 하나로 작동한다고 단정하면 안 된다. 공개 lex.rs

코드 읽기 과제:

1. add_document를 검색하고 입력값이 무엇인지 적는다.

2. finish에서 메모리 안의 자료를 어떻게 저장 가능한 바이트로 바꾸는지 찾는다.

3. search에서 질문이 결과 목록으로 바뀌는 단계를 적는다.

4. frame_id가 왜 결과에 남는지 설명한다.

5. 파일 이름에 lex가 있다고 벡터 모델 호출 코드까지 있다고 기대하지 않는다.

README의 "BM25 지원"과 특정 모듈에서 보이는 단순 문자열 일치 코드는 동일한 수준의 설명이 아닐 수 있다. 설치된 SDK가 어떤 코어·기능·검색 경로를 사용하는지는 버전까지 연결해서 확인해야 한다. 초보자는 일단 API 호출과 관찰한 결과를 정확히 기록하는 것부터 시작하면 된다.

14. 사건별 파일 격리를 제대로 이해하기

추천하는 첫 학습 구조는 사건별 파일이다.

```
서버가 관리하는 메모리 루트/
  사무실A/사건001.mv2
  사무실A/사건002.mv2
  사무실B/사건003.mv2
```

사용자가 보낸 문자열을 그대로 파일 경로로 쓰지 않는다. Spring에서 로그인 사용자와 사건 접근 권한을 확인하고, 서버가 관리하는 매핑으로 파일을 고른다. Python에도 인증된 내부 요청만 전달한다.

case_id만 맞으면 읽도록 만드는 것은 부족하다. 권한 없는 사용자가 다른 사건 ID를 알아낼 수 있기 때문이다. 파일별 분리는 실수를 줄이는 구조이며, 접근 권한의 대체물이 아니다.

추가로 검토할 것:

- 운영체제 권한: 누가 파일을 읽거나 복사할 수 있는가.

- 백업과 로그: 파일 밖에 원문이나 검색 결과가 남는가.

- 암호화와 키 관리: 파일 유출 시 내용을 읽을 수 있는가.

- 삭제: 프레임 삭제 후에도 파일의 이전 기록·백업에 내용이 남는가.

- 동시 쓰기: 두 작업이 같은 사건에 동시에 저장하면 어떻게 되는가.

공식 ACL 문서는 메타데이터와 조회 시 권한 문맥을 함께 설정하는 방식을 설명한다. 단순 라벨이나 임의의 case_id 저장과 다르다. 운영에서는 파일 분리와 애플리케이션 권한 검증을 먼저 명확히 하고, ACL 추가 적용 여부를 검증한다. 권한 기반 검색

15. 우리 Spring Boot-FastAPI 구조에 연결하기

다음은 기존 설계를 공부하기 위한 연결 지도다. 이 학습 파일이 운영 서버를 수정한 것은 아니다.

> **흐름도 안내**
>
> 이 Mermaid 원문의 처리 흐름은 앞의 TOP 전체 지도에 벡터 도식으로 반영했습니다. 상세 단계는 이 장의 설명과 함께 읽습니다.

15.1 Spring: 원문과 정책 통제

Spring은 인증·사건 소유권·입력 크기·세션 정책·예산을 통제한다. PII를 전부 같은 [마스킹]으로 바꾸면 당사자를 구별하기 어려워진다. [PERSON_001], [PERSON_002] 같은 사건 내 일관된 치환 표식을 사용할 수 있다. 원문 복원 매핑은 접근 통제된 별도 영역에 둔다.

정규식은 주민번호처럼 형식이 있는 정보의 1차 탐지에는 유용하지만 모든 이름과 자유 서술의 민감정보를 잡는다고 보장할 수 없다. 로그와 예외 메시지도 마스킹 경계를 지키는지 확인한다.

15.2 Python: 검색과 생성 분리

FastAPI가 받을 학습용 요청 계약 예시는 다음과 같다.

```
{
  "request_id": "REQ-DEMO-001",
  "case_id": "CASE-DEMO-001",
  "masked_query": "보증금 반환 상담 자료를 정리해 주세요",
  "max_context_tokens": 2000,
  "policy_version": "intake-v2"
}
```

이 필드들은 memvid가 자동으로 이해하는 공통 규격이 아니라 우리 서비스가 정하는 API다. 사건 ID·한도·정책 버전은 인증된 Spring 문맥에서 받아야 한다. 클라이언트가 바꿔 보낸 max_context_tokens를 그대로 신뢰하면 안 된다.

처음에는 이 요청을 받아 02_search.py와 같은 근거 JSON을 반환하는 단계까지만 만든다. 이후에 토큰 계산, 비용 승인, 모델 호출, 출력 스키마 검증을 붙인다. 저장과 조회를 모두 하나의 공개 API에서 무제한 수행하도록 시작하지 않는다.

Llama는 모델 계열 이름이고 Gemini도 모델 계열 이름이다. "CEO 모델"은 역할 이름이다. 실제 모델 ID와 담당 역할을 구분해서 설정한다. 두 모델을 항상 연달아 호출할 필요는 없다. 간단한 입력은 저비용 경로로 처리하고, 복잡하거나 근거가 충돌하는 경우만 상위 검토 경로로 보내는 정책을 평가한다.

15.3 Spring: 최종 검증과 화면 저장

Spring은 허용 필드·출력 길이·정책 위반·PII 재노출·출처 연결 등을 검사하고 검토 상태로 저장한다. 정적 규칙으로 모든 환각을 검출할 수는 없다. 관리자 화면에는 원문 출처와 미확인 항목을 함께 보여줘야 한다.

claim_summary, evidence, timeline, missing_information 같은 구조는 화면에서 별도 영역으로 매핑하기 좋다. 보고서와 검색 파일을 동일한 저장소로 취급하지 않는다. 사건·권한·보고서 상태·비용 원장은 기존 관계형 DB가 담당하고, memvid는 검색용 메모리를 담당하도록 나눌 수 있다.

16. 비용 관리 여섯 가지를 연결하는 연습

아래는 연결 학습을 위한 **제안 분류**다. 현재 저장소에 동일한 여섯 항목이 구현되어 있다고 확인한 내용은 아니다.

관리 항목	검사 시점	예시
요청 크기	입력 수신	지나치게 긴 입력 제한
호출 빈도	요청 승인 전	사용자별 짧은 시간의 반복 요청 제한
세션 누적 점수	각 요청	누적 사용·위험 신호를 정책 점수로 평가
예산 예약	LLM 직전	동시에 온 요청이 같은 잔액을 중복 사용하는 것 방지
호출·출력 제한	모델 호출 중	출력 상한·타임아웃·재시도 횟수 제한
사용량 정산	완료·실패·취소 후	모델 응답의 사용량을 모아 예약분과 정산

점수, 토큰, 금액은 서로 다른 단위다. Cumulative Score는 정책 판단 값이고, 토큰 수는 모델 사용량이며, 금액은 요금표를 적용한 결과다. 점수를 곧바로 정확한 원가라고 부르지 않는다.

사용량에 포함할 항목은 입력·출력뿐 아니라 임베딩, 중간 모델, 재시도, 캐시·추론 관련 과금 등 실제 제공자의 항목을 따른다. 사용량을 못 받은 실패 호출은 UNKNOWN 같은 상태로 남겨 추정치와 확정치를 분리한다.

17. 혼자 하는 검색 품질 평가

빠르게 검색된다는 것과 올바른 근거를 찾는다는 것은 다르다. 우선 가상 사건 5개, 사건당 자료 5개, 질문 20개처럼 손으로 검토 가능한 작은 평가집을 만든다. 숫자는 연습용 시작 규모다.

질문 유형	확인할 점
정확한 단어	보증금·금액·날짜가 있는 자료를 찾는가
다른 표현	돌려받기와 반환을 연결하는가
부정 표현	제출하지 않았다를 제출했다로 바꾸지 않는가
충돌 자료	두 진술이 다르면 둘 다 보여주는가
자료 없음	내용을 만들어내지 않고 추가 자료를 요청하는가
다른 사건	다른 의뢰인의 자료가 섞이지 않는가

질문마다 정답 문서 ID를 사람이 먼저 정한다. 모델 답변을 보고 나중에 정답을 바꾸지 않는다. 예를 들어 관련 문서가 D001과 D002 두 개이고 상위 3개 결과에 D001만 포함됐다면 그 질문의 Recall@3은 1/2이다.

함께 기록할 값은 검색 시간, 전체 응답 시간, 실제 토큰 수, 호출 수, 근거 누락, 사건 간 누출 여부다. LLM이 문장을 자연스럽게 썼는지는 검색 평가와 별도로 본다.

기존 RAG와 비교할 때는 같은 자료·질문·정답·임베딩·분할 조건을 최대한 맞춘다. 검색 엔진과 임베딩 모델을 동시에 바꾸면 어느 변경이 효과를 냈는지 해석하기 어렵다. 이 평가 후에 "우리 인테이크 규모에서는 충분하다"는 결론을 내린다.

18. 오류가 나면 확인할 순서

증상	먼저 볼 곳	다음 행동
py를 찾지 못함	Python 설치·PATH	사용 가능한 Python 실행 파일 확인
ModuleNotFoundError	설치한 환경과 실행한 환경	같은 .venv의 Python으로 pip와 스크립트 실행
배포 파일 설치 실패	OS·CPU·Python 조합	지원 wheel과 설치 오류 확인
기존 파일 안내	01을 이미 실행했는가	정상 동작이다. 02로 이동
검색 결과가 없음	실제 저장 문장·키워드	저장한 단어 하나로 검색
의미 검색 인덱스 오류	저장 시 벡터 생성 여부	새 실습 파일에서 저장부터 다시 확인
Windows 로컬 임베딩 실패	기본 임베딩 지원 범위	키워드 실습 완료 후 지원 환경으로 이동
파일 잠금 오류	동시에 실행한 프로세스	파일을 사용하는 작업을 정상 종료
용량 초과	SDK 용량 설정과 정책	상태 확인·자료량 축소·지원 정책 검토
응답 필드 오류	고정 버전과 실제 응답	원본 JSON에서 필드 확인 후 변환기 수정

잘 안 된다고 계속 create()를 호출하거나 데이터 파일을 즉시 삭제하지 않는다. 원인과 파일 상태부터 확인한다. 오류를 질문할 때는 명령, 버전, 오류 메시지를 남기되 개인정보와 API 키는 빼고 공유한다.

19. 7일 자습 일정

일차	할 일	끝났다고 판단할 기준
1일	1~5장 읽기·가상환경 준비	RAG와 저장 엔진의 차이를 설명함
2일	01 저장 코드 읽고 실행	파일 생성과 반복 생성 방지를 확인함
3일	02 검색 실행·변수 추적	결과 전체·결과 하나·문자열을 구분함
4일	근거 길이·JSON 이해	글자 제한과 토큰 제한의 차이를 설명함
5일	지원 환경에서 선택 의미 검색	재개방 후 검색하고 언어 한계를 기록함
6일	Spring-FastAPI 계약 작성	권한·마스킹·예산 검사 위치를 표시함
7일	작은 평가집 만들기	정답 문서와 검색 결과를 비교함

각 날짜의 결과를 "잘 됨" 한 줄로 적지 말고 입력 질문, 예상 출처, 실제 출처, 배운 점으로 나눠 기록한다. 실행에 막히면 일정을 늘려도 된다. 키워드 검색을 이해한 상태에서 의미 검색으로 넘어가는 것이 목표다.

20. 이해 확인 문제와 답

1. memvid에 저장하면 LLM 학습 가중치가 바뀌는가? → 아니다. 저장 자료를 나중에 검색해 컨텍스트로 사용하는 것이다.

2. .mv2 하나면 모든 비용이 사라지는가? → 아니다. 실행 자원과 임베딩·생성·운영 비용은 따로 본다.

3. 사건별 파일이면 인증이 필요 없는가? → 아니다. 파일 선택 전에 사용자와 사건 권한을 검사한다.

4. enable_embedding=False의 의미는? → 이번 입력에서 임베딩 생성을 사용하지 않는 설정이다.

5. k=3이면 왜 세 개보다 적게 나올 수 있는가? → 조건에 맞는 검색 결과가 충분하지 않을 수 있다.

6. JSON 파싱 성공이 왜 법률적 정확성을 뜻하지 않는가? → 형식 검증과 근거·내용 검증이 다르기 때문이다.

7. 검색 결과가 없다면? → 근거 부족 상태로 처리하고 추가 정보를 요청한다.

8. 단어 검색과 의미 검색 중 무조건 더 정확한 것은? → 없다. 질문·언어·자료·모델에 따라 평가한다.

9. 모델을 바꿀 때 차원 수만 맞으면 되는가? → 아니다. 같은 임베딩 공간과 설정이 필요하다.

10. 현재 우리가 가장 먼저 성공시킬 기능은? → 가상 자료 저장 → 종료 → 재개방 → 검색 → 근거 JSON이다.

21. 다음에 설계 수정안 2로 옮길 문장

설계 문장 / 설명 예시

Spring Boot가 사건 권한·PII 처리·비용 정책을 통제하고, Python FastAPI가 memvid 기반 사건별 검색으로 필요한 근거를 선택한다. 모델 호출 직전 최종 예산을 확인하고, 생성 결과의 구조와 출처를 검증한 뒤 Spring에서 관리자 검토용 보고서로 저장한다. 검색 품질·격리·비용 절감은 평가로 확인한다.

이 문장을 설계의 기준으로 삼으면 단일 파일의 장점을 살리면서 비용 통제와 출처 검증을 유지할 수 있다.

22. 변호사에게 설명할 세 가지 가치: 보안·검토 품질·예측 가능한 비용

이 장은 사용자가 제안한 세 가지 장점을 기술적으로 보강한 내용이다. **아래는 구현 목표와 검증 방법이며, 현재 실습 코드에서 완성된 기능 목록이 아니다.** 특히 01_store.py가 만드는 .mv2는 암호화 파일이 아니다. 기존 3개 실습에 암호화·사용자 인증·실제 비용 차단 기능이 추가된 것으로 이해하지 않는다.

22.1 첫 번째 가치: 사건 단위로 접근·보관·반출 범위를 관리하기 쉽다

변호사가 알고 싶은 것은 "벡터 DB를 쓰는가"보다 **"어느 의뢰인의 자료를 누가 읽고, 어떤 외부 서비스에 보내며, 어떻게 폐기하는가"**다. 사건별 파일은 이 관리 단위를 코드와 운영 절차에서 명확히 잡는 데 도움이 된다.

예를 들어 사건 A를 처리할 때 사건 A의 메모리만 열도록 만들면, 하나의 공유 검색 공간에서 매번 사건 필터를 정확히 적용해야 하는 부담을 줄일 수 있다. 사건 단위의 백업·이관 대상도 구분하기 쉽다. 다만 같은 서버의 파일을 나눈 것은 **파일 단위 분리**다. 별도 하드웨어·프로세스·스토리지까지 분리했다는 뜻의 물리적 격리와 구분한다.

중앙 벡터 DB도 올바른 권한과 테넌트 분리로 보호할 수 있다. 반대로 개별 파일도 서버 권한이 잘못되거나 파일 선택 코드가 틀리면 유출될 수 있다. 비교의 초점은 저장 방식 자체의 우열보다 우리 팀이 접근 경계를 얼마나 명확히 구현하고 검증할 수 있는지다.

기술적으로 필요한 네 가지 경계

경계	역할	적용 예
사용자 → 사건	이 사용자가 해당 사건을 볼 수 있는가	Spring에서 담당자·사무실·사건 권한 확인
사건 → 파일	승인된 사건이 올바른 파일로 연결되는가	서버의 사건-파일 매핑, 임의 경로 입력 금지
파일 → 평문	저장 매체 유출 시에도 읽을 수 없게 하는가	암호화, 별도 비밀 관리, 복호화 작업 공간 통제
검색 결과 → 외부 모델	어떤 근거가 어디로 나가는가	PII 처리, 외부 전송 정책, 승인된 모델 경로

공식 암호화 문서는 AES-256-GCM과 Argon2id 기반 .mv2e 캡슐을 안내한다. 확인한 문서의 흐름에서는 검색 전에 복호화가 필요하다. 따라서 "암호화된 상태로 그대로 검색한다"고 설명하지 않는다. 암호화 사용 가능 여부와 API는 도입할 SDK·CLI 빌드에서 별도로 확인한다. 공식 암호화 안내

초보자는 다음 개념 흐름으로 이해하면 된다. 실행 명령이나 구현 완료를 뜻하는 그림은 아니다.

```
사건 접근 권한 확인
  → 서버 매핑으로 파일 선택
  → 별도 비밀 저장소에서 해당 사건의 복호화 자격 확보
  → 통제된 작업 공간에서 복호화·검색
  → 선택 근거만 처리
  → 변경이 있으면 암호화된 새 저장본을 검증하고 교체
  → 작업용 평문과 임시 산출물의 수명 관리
```

사건별 키를 분리하면 특정 키가 노출되었을 때의 영향을 줄이는 설계를 할 수 있다. 그러나 memvid가 사건별 키 발급·회전·복구를 자동으로 운영해 준다고 가정하면 안 된다. 이는 별도 애플리케이션·비밀 관리 설계다. 파일과 비밀번호를 같은 폴더에 두면 파

일 암호화의 이점이 크게 줄어든다.

복호화된 자료는 검색 중 프로세스 메모리나 작업 파일에 존재할 수 있다. 비정상 종료 시 남는 임시 파일, 운영체제 스왑, 장애 덤프, 백업까지 고려해야 한다. 단순 파일 삭제만으로 저장 매체에서 완전 삭제되었다고 보장하지 않는다. 단일 파일 전체를 매 요청마다 복호화·재암호화하면 지연과 디스크 비용이 늘 수 있으므로 처리 단위와 동시 실행 정책도 측정한다.

SDK의 ACL을 추가할 경우 권한 문맥은 인증된 서버 정보로 만들고 실제 차단 모드로 적용해야 한다. 감사만 하는 모드는 접근을 막지 않는다. 공식 ACL 안내

검증할 사례: 사건 A 담당자가 사건 B의 ID를 요청하면 파일 열기 전에 거부되는지, 오류 로그에 평문이 남지 않는지, 암호화 백업만으로 정상 복원이 되는지, 권한 해제 후에도 캐시가 자료를 반환하지 않는지 확인한다. 같은 의뢰인의 여러 사건도 필요에 따라 권한을 분리한다.

변호사에게 설명할 문장:

> **설계 문장 / 설명 예시**
>
> 사건별로 검색 메모리를 나누고, 담당자 권한·저장 암호화·외부 전송 정책을 함께 적용해 자료 접근 범위를 관리하도록 설계합니다. 접근 기록과 복원 검증으로 운영 상태를 확인합니다.

이 설명은 기술적 보호 조치에 관한 것이다. 특정 법률상 의무를 충족했다는 법적 판단을 대신하지 않는다.

22.2 두 번째 가치: 원문을 확인할 수 있는 요약으로 검토 부담을 줄인다

변호사에게 실질적인 가치는 **보고서의 문장을 원문과 대조하고, 미확인 항목을 빠르게 찾을 수 있는 것**이다. memvid는 필요한 근거를 꺼내는 역할을 맡고, 애플리케이션은 근거와 보고서를 연결한다.

단일 파일이나 인프라 단순화 자체가 환각을 줄이는 직접적인 근거는 아니다. 품질은 자료 추출·검색·근거 선택·모델 생성·검토의 전체 흐름에 달려 있다. 관련 문단을 놓치거나 OCR이 금액을 잘못 읽었다면 가벼운 저장 엔진도 잘못된 내용을 제공할 수 있다.

요약 정확도를 높이는 처리 흐름

1. **자료 식별:** 원본 ID·버전·페이지·청크 위치를 남긴다. 원본 해시는 변경 확인에 도움이 되지만 내용의 진실성을 증명하지는 않는다.

2. **사건 범위 제한:** 승인된 사건의 자료에서만 근거를 찾는다. 공통 법령·절차 자료를 쓴다면 사건 진술과 별도 출처로 구분한다.

3. **근거 선택:** 문단 단위로 선택하고 날짜·금액·부정 표현·상충 진술을 보존한다. 검색 결과에 포함된 명령문은 자료로 취급한다.

4. **출처 연결 생성:** 보고서의 주요 주장에 source_id와 페이지 또는 청크 범위를 요구한다. 근거가 없는 부분은 미확인으로 남긴다.

5. **기계 검증:** JSON 형식, 출처 존재 여부, 사건 일치 여부, 수치·날짜 일치 여부를 검사한다. 출처가 존재해도 그 문장이 주장을 실제로 뒷받침하는지는 별도 검토한다.

6. **변호사 검토:** 보고서에서 원문으로 이동할 수 있게 하고, 충돌·누락·확인 필요 항목을 표시한다. 사람의 수정은 승인된 이력으로 남긴다.

예를 들어 연락 메모에 "의뢰인이 반환을 요청했다고 진술했다"고만 적혀 있다면, 보고서는 이를 **의뢰인 진술**로 표시해야 한다. 이를 "상대방이 반환 요청을 수령했다"로 바꾸면 근거의 범위를 넘어선다. 이 구분은 문장을 매끄럽게 만드는 것보다 중요하다.

최종 보고서의 근거 연결을 확장하는 예시다. 이 역시 우리 서비스가 정의할 계약이며 memvid 기본 응답 형식이 아니다.

```
{
  "statement": "의뢰인은 2026-10-05 반환 요청을 했다고 진술함",
  "basis_type": "CLIENT_STATEMENT",
  "sources": [
    {"source_id": "D002", "chunk_id": "D002-C01"}
  ],
  "verification_status": "NEEDS_DOCUMENT_CONFIRMATION"
}
```

공식 SDK 문서에는 답변과 근거의 연결 정도를 나타내는 grounding 정보가 소개되어 있다. 이는 검토에 쓸 수 있는 신호이며, 높은 점수가 법률적 정확성이나 무환각을 증명하지는 않는다. 별도 LLM 호출 경로를 만든 경우에도 동일 정보가 자동으로 생긴다고 가정하지 않는다. 공식 SDK 안내

평가할 지표: 근거 없는 주장 비율, 출처 누락 비율, 날짜·금액 추출 오류, 상충 진술 누락, 변호사의 수정 시간이다. 수정 시간이 줄었는지는 같은 자료와 유사한 검토 조건에서 실제로 측정한다. 기존 방식보다 우수하다는 표현도 비교 평가 이후에 쓴다.

변호사에게 설명할 문장:

> **설계 문장 / 설명 예시**
>
> 제출 자료의 출처를 연결한 상담 요약을 제공하고, 확인되지 않은 진술과 누락 자료를 구분해 변호사가 검토할 부분을 빠르게 찾도록 돕습니다.

22.3 세 번째 가치: 별도 검색 인프라를 줄이고 요청별 지출을 관리한다

비용 측면의 장점은 두 부분으로 나뉜다. memvid는 **별도 검색 DB 서비스의 운영 항목을 줄일 가능성**이 있고, Spring의 가드레일은 **모델 호출의 승인·한도·정산을 관리**한다. 두 역할을 함께 설명해야 절감 원인이 명확하다.

비용을 빠뜨리지 않고 나누는 법

```
월 총운영비
  = Spring·FastAPI 실행 자원
  + 관계형 DB·스토리지·암호화 백업
  + 문서 추출·OCR·임베딩
  + LLM 입력·출력 및 제공자별 추가 과금
  + 모니터링·복구·운영 인력
  + 사용하는 SDK·클라우드 플랜 비용
```

단일 파일도 디스크와 백업 공간을 사용한다. 로컬 모델은 외부 API 요금을 줄일 수 있지만 CPU·메모리와 실행 시간을 소비한다. 소규모에서 검색 DB를 없애 절감한 금액보다 운영 복잡도 감소가 더 큰 이점일 수도 있다. 어느 쪽이든 사용량을 측정해 판단한다.

비용 예약이 필요한 이유

잔여 예산이 100원인데 동시에 요청 두 개가 각각 예상 비용 80원을 검사한다고 해보자. 두 요청이 모두 "잔액 100원"을 읽으면 각각 통과해 합계 160원까지 승인할 수 있다. 따라서 잔액 조회와 비용 예약은 하나의 원자적 처리로 묶어야 한다.

```
request_id로 중복 요청 확인
  → 최종 입력·출력 상한과 모든 예정 호출 비용 추정
  → 잔여 예산 검사와 예약을 원자적으로 수행
  → 승인된 모델·출력 상한·재시도 한도 내에서 실행
  → 실제 사용량 수집
  → 예약분을 확정 사용량으로 정산
```

이 흐름은 설계 예시다. 현재 Spring 구현이 이 기능을 모두 갖추었다고 확인한 설명은 아니다. 상위 모델 재검토·요약용 중간 호출·임베딩·재시도도 승인 범위에 넣어야 한다. 모델 호출을 다른 서비스에서 우회할 수 있다면 중앙 비용 통제가 성립하지 않는다.

"실시간 완벽 차단"도 정확히 구분한다. 요청 전 차단과 출력 상한 설정은 가능하지만, 취소 요청이 도착하기 전에 공급자가 이미 처리한 토큰 비용까지 되돌릴 수 있는 것은 아니다. 타임아웃 후에도 공급자 처리가 진행 중일 수 있으므로 즉시 예약을 전부 해제하지 않고 미확정 상태로 추적하는 정책이 필요하다.

글자 수는 초기 입력 크기 검사에 쓸 수 있지만 최종 금액 계산의 기준은 아니다. 모델별 토큰 계산과 실제 공급자 사용량을 기준으로 원장을 만들고, 사용량을 받지 못한 요청은 추정치와 확정치를 분리한다. memvid 계정의 쿼리 계량과 외부 LLM 과금도 서로 다른 항목이다. memvid 사용량 안내

검증할 사례: 잔액에 가까운 동시 요청, 같은 요청의 재전송, 모델 타임아웃, 중간 단계 실패, 상위 모델 전환, 스트리밍 취소에서 예약·정산이 맞는지 확인한다. 가드레일의 정확성은 단계 수보다 이 경계 상황에서의 동작으로 평가한다.

변호사에게 설명할 문장:

설계 문장 / 설명 예시

별도 검색 DB 운영을 줄이는 구성을 적용하고, 사용자·사건·기간별 모델 사용 한도와 요청별 비용 기록으로 지출을 관리하도록 설계합니다. 실제 사용량을 바탕으로 포함량과 초과 요금을 정할 수 있습니다.

가격은 정액 기본료와 포함 사용량, 추가 처리 요금을 조합할 수 있다. 다만 요금의 매력과 계약 성사 여부는 실제 원가·검토 시간 절감·지원 범위·고객 수요를 확인해 판단한다. 기술 선택만으로 판매 성공이 보장되지는 않는다.

22.4 세 가지 가치를 도입 판단 자료로 정리하기

제안 가치	뒷받침할 기술	고객에게 제시할 증거
사건별 접근 범위 관리	사건-파일 매핑, 권한 검증, 암호화·키 관리	교차 사건 접근 차단·복원·권한 해제 검증 결과
검토하기 쉬운 요약	출처 연결, 진술·확인 사실 구분, 누락·충돌 표시	근거 오류율, 검토자 수정 내역, 검토 시간 비교
예측 가능한 지출	검색 인프라 단순화, 예산 예약, 한도, 사용량 정산	요청별 원가, 동시 요청 차단 결과, 월 사용량 보고서

학습자는 세 장점을 각각 "무엇을 구현할 것인가 → 어떤 오류를 막는가 → 어떻게 확인할 것인가"로 설명해 보자. 이 연결이 갖춰지면 홍보 문구를 실제 설계와 운영 증거로 뒷받침할 수 있다.

23. 직접 확인한 참고자료

공식 예제를 그대로 길게 옮기기보다 가상 인테이크 학습용 코드로 다시 구성했다. 실습 파일의 API와 설명은 위 각 장의 링크를 함께 보면서 읽는다.

- 공식 저장소와 Rust 시작 예제
- 버전 변경 이력
- 공식 5분 시작 안내
- Python 설치
- Python API 설명
- Python 조회 기능
- PyPI 배포 파일과 플랫폼 안내
- 임베딩 모델 설명
- Rust 모듈 입구
- Rust 단어 검색 코드
- 권한 기반 검색
- 암호화 캡슐과 복호화 흐름
- 용량 안내
- 쿼리 사용량 안내

코드 01 · 01_store.py (1/2)

전체 파일 1-17줄. 줄 번호는 읽기용입니다. 구문 분석만 확인했으며 SDK 동작은 미검증입니다.

```
01 """가상 인테이크 자료 저장. 실제 개인정보를 넣지 마세요."""
02 import os
03 from pathlib import Path
04
05 # 이번 입문 실습은 네트워크 기능과 자동 임베딩을 사용하지 않습니다.
06 os.environ["MEMVID_OFFLINE"] = "1"
07
08 from memvid_sdk import create
09
10 path = Path(__file__).resolve().parent / "data" / "case-demo.mv2"
11 path.parent.mkdir(parents=True, exist_ok=True)
12
13 # create()의 기존 파일 덮어쓰기를 막고, 반복 실행 시 중복 입력도 막습니다.
14 if path.exists():
15     raise SystemExit("실습 파일이 이미 있습니다. 02_search.py로 검색하세요.")
16
17 documents = [
```

코드 01 · 01_store.py (2/2)

전체 파일 18-34줄. 줄 번호는 읽기용입니다. 구문 분석만 확인했으며 SDK 동작은 미검증입니다.

```
18     ("D001", "계약 메모", "보증금 1000만원. 계약 종료 예정일은 2026-09-30이다."),
19     ("D002", "연락 메모", "2026-10-05 의뢰인이 보증금 반환을 요청했다고 진술했다."),
20     ("D003", "자료 목록", "계약서 사본이 있고, 송금 내역은 아직 제출되지 않았다."),
21 ]
22
23 with create(str(path), enable_lex=True, enable_vec=False) as memory:
24     for source_id, title, text in documents:
25         memory.put(
26             title=title,
27             label="intake-demo",
28             metadata={"source_id": source_id, "case_id": "case-demo", "synthetic": True},
29             text=text,
30             enable_embedding=False,
31         )
32     memory.seal()
33
34 print(f"가상 문서 {len(documents)}개를 저장했습니다: {path}")
```

코드 02 · 02_search.py (1/3)

전체 파일 1-15줄. 줄 번호는 읽기용입니다. 구문 분석만 확인했으며 SDK 동작은 미검증입니다.

```
"""검색 결과 원본과 길이를 제한한 근거 묶음을 확인합니다. LLM 호출 없음."""
import json
import os
import sys
from pathlib import Path

os.environ["MEMVID_OFFLINE"] = "1"

from memvid_sdk import use

path = Path(__file__).resolve().parent / "data" / "case-demo.mv2"
if not path.exists():
    raise SystemExit("01_store.py를 먼저 실행하세요.")

query = sys.argv[1] if len(sys.argv) > 1 else "보증금"
```

코드 02 · 02_search.py (2/3)

전체 파일 16-30줄. 줄 번호는 읽기용입니다. 구문 분석만 확인했으며 SDK 동작은 미검증입니다.

```
16 with use("basic", str(path)) as memory:
17     result = memory.find(query, k=3, mode="lex", snippet_chars=500)
18
19 # SDK 버전의 실제 반환 필드를 먼저 확인합니다.
20 print("검색 응답 원본:")
21 print(json.dumps(result, ensure_ascii=False, indent=2, default=str))
22
23 hits = result["hits"]
24 evidence = []
25 remaining = 1200  # 글자 수 상한이지 토큰 상한이 아닙니다.
26 for hit in hits:
27     text = hit.get("snippet") or hit.get("preview") or hit.get("text") or ""
28     if not isinstance(text, str) or not text or remaining <= 0:
29         continue
30     selected = text[:remaining]
```

코드 02 · 02_search.py (3/3)

전체 파일 31-44줄. 줄 번호는 읽기용입니다. 구문 분석만 확인했으며 SDK 동작은 미검증입니다.

```
31      evidence.append({
32          "frame_id": hit.get("frame_id"),
33          "title": hit.get("title"),
34          "text": selected,
35      })
36      remaining -= len(selected)
37
38  print("₩nLLM에 전달하기 전 검토할 근거 묶음:")
39  print(json.dumps({
40      "query": query,
41      "status": "EVIDENCE_FOUND" if evidence else "NEEDS_MORE_INFORMATION",
42      "evidence": evidence,
43      "requires_lawyer_review": True,
44  }, ensure_ascii=False, indent=2))
```

코드 03 · 03_semantic_optional.py (1/2)

전체 파일 1-18줄. 줄 번호는 읽기용입니다. 구문 분석만 확인했으며 SDK 동작은 미검증입니다.

```
01 """선택 실습: Linux/macOS 기본 로컬 임베딩 경로. Windows 기본 실습과 분리."""
02 import json
03 import os
04 import sys
05 from pathlib import Path
06
07 if sys.platform == "win32":
08     raise SystemExit("기본 로컬 임베딩 실습은 Linux/macOS에서 진행하세요. 학습서 11장 참고.")
09
10 # 외부 API 자동 선택을 피합니다. 첫 실행에는 로컬 모델 다운로드가 필요할 수 있습니다.
11 if os.environ.get("OPENAI_API_KEY") or os.environ.get("MEMVID_API_KEY"):
12     raise SystemExit("외부 API 키가 없는 별도 실습 터미널에서 실행하세요.")
13
14 from memvid_sdk import create, use
15
16 path = Path(__file__).resolve().parent / "data" / "semantic-demo.mv2"
17 path.parent.mkdir(parents=True, exist_ok=True)
18
```

코드 03 · 03_semantic_optional.py (2/2)

전체 파일 19-35줄. 줄 번호는 읽기용입니다. 구문 분석만 확인했으며 SDK 동작은 미검증입니다.

```
19 if not path.exists():
20     with create(str(path), enable_vec=True, enable_lex=True) as memory:
21         memory.put(
22             title="Synthetic rental note",
23             label="demo",
24             metadata={"source_id": "S001", "synthetic": True},
25             text="The tenant asked the landlord to return the rental deposit.",
26             enable_embedding=True,
27             embedding_model="bge-small",
28         )
29         memory.seal()
30
31 with use("basic", str(path)) as memory:
32     result = memory.find(
33         "getting the deposit back", k=3, mode="sem", query_embedding_model="bge-small"
34     )
35     print(json.dumps(result, ensure_ascii=False, indent=2, default=str))
```